操作系统

进程

周炽金

华东师范大学
软件工程学院

这一章回答哪些问题?

- **程序运行后，系统管理什么?** 进程、地址空间与 PCB。
- **多个进程怎样轮流运行?** 状态、队列与上下文切换。
- **进程怎样产生和结束?** fork、exec、wait 与退出状态。
- **进程怎样交换数据?** 共享内存、消息与管道。
- **通信怎样延伸到另一台机器?** Socket 与 RPC。

资料：[1] Operating System Concepts，第 3 章

程序与进程有什么区别?

*A process is a **program in execution**.*

- **程序（Program）**：磁盘上的指令与数据，是静态文件。
- **进程（Process）**：程序的一次执行，具有运行状态和资源。
- 同一个程序可以对应多个进程，各自有标识和执行进度。
- 例如打开两个独立的终端，可能运行同一 Shell 程序的两个实例。

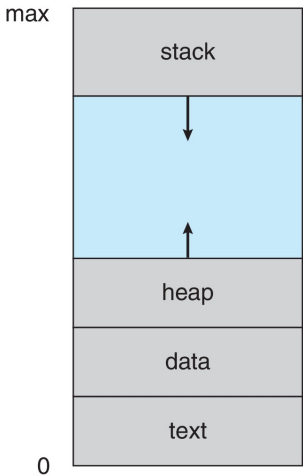
资料：[1] Operating System Concepts，第 3 章

一个进程包含什么?

- **执行进度**：程序计数器 PC、寄存器等 CPU 状态。
- **代码与数据**：代码段、全局和静态数据。
- **栈（Stack）**：函数调用所需的局部数据、参数和返回信息。
- **堆（Heap）**：运行时动态申请的内存。
- **系统资源**：打开的文件、身份、通信端点等。

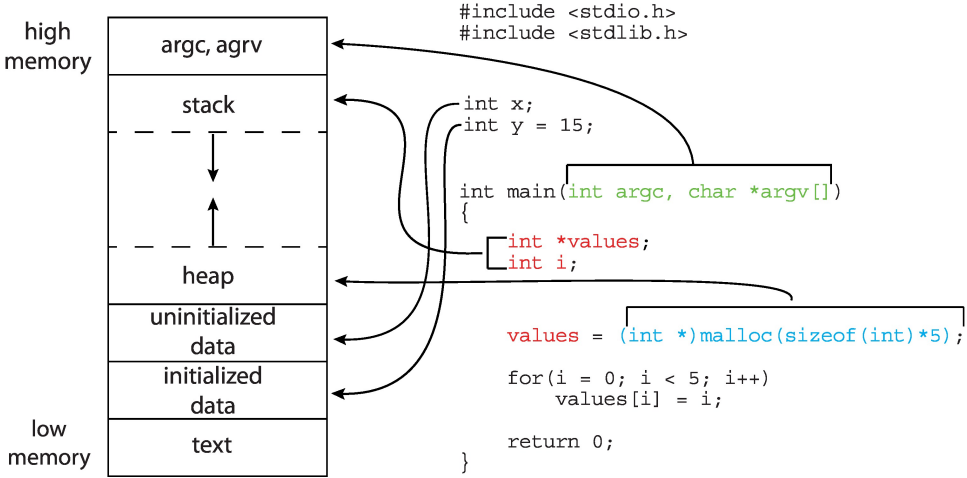
资料：[1] Operating System Concepts，第 3 章

进程的地址空间



资料：[1] Operating System Concepts，第 3 章

C 程序中的数据放在哪里？



资料：[1] Operating System Concepts，第 3 章

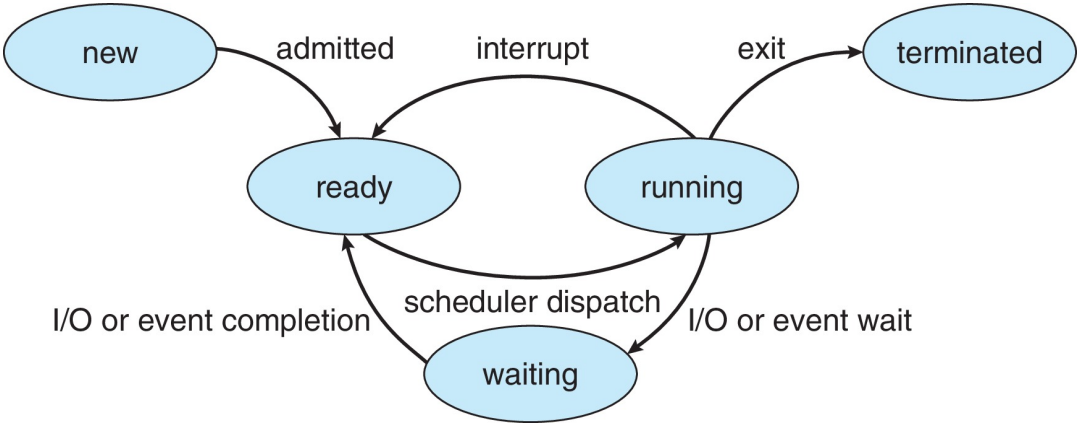
进程有哪些状态?

状态	含义
New（新建）	系统正在创建进程
Ready（就绪）	已具备执行条件，等待 CPU
Running（运行）	正在 CPU 上执行
Waiting（等待）	等待 I/O、消息或其他事件
Terminated（终止）	执行已结束，进入清理和回收阶段

就绪是在等 CPU；等待是在等某个事件。

资料：[1] Operating System Concepts，第 3 章

状态怎样转换?



资料：[1] Operating System Concepts，第 3 章

系统怎样记住一个进程?

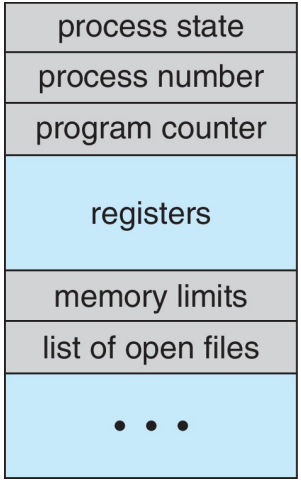
PCB（Process Control Block）保存管理和恢复进程所需的信息。

- **标识与状态**：PID、父子关系、运行状态。
- **执行上下文**：恢复执行所需的寄存器、PC 等信息。
- **调度与内存信息**：优先级、队列关系、地址空间。
- **资源与统计**：打开的文件、CPU 使用量、权限等。

PCB 是内核的数据结构；进程不在 CPU 上运行时，它仍然存在。

资料：[1] Operating System Concepts，第 3 章

PCB 的字段结构



资料：[1] Operating System Concepts，第 3 章

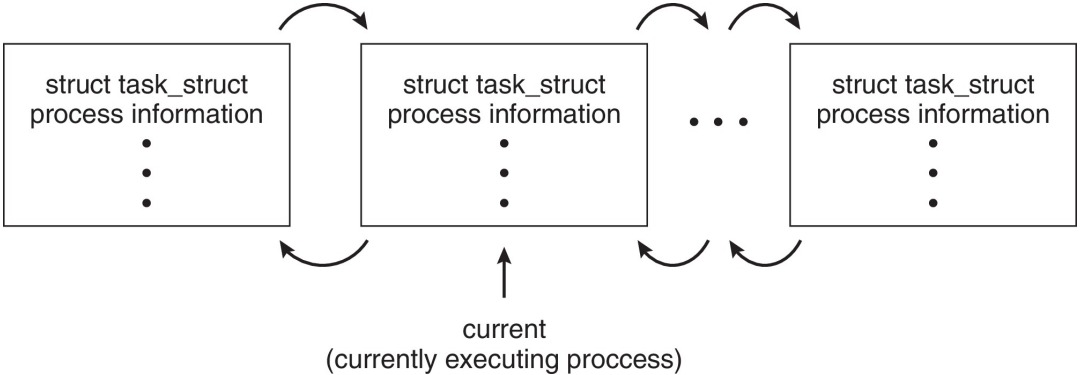
在 Linux 中观察进程

```
ps -o pid,ppid,stat,comm -p $$
cat /proc/$$/status
```

- Linux 用 **task_struct** 等结构管理任务及其相关资源。
- 用户可通过 `/proc` 观察 PID、父进程、状态、线程数和内存信息。
- `R` 包含正在运行与可运行；`S` 常表示可中断睡眠，`Z` 表示僵尸。
- 观察的是瞬时状态，连续两次读取可能不同。

资料：[5] proc_pid_status(5)　[1] Operating System Concepts，第 3 章

Linux 任务链表与当前任务



资料：[1] Operating System Concepts，第 3 章

一个进程是否只能做一件事?

- 前面的模型先假设每个进程只有**一个执行线程**。
- 一个进程可以包含多个线程，各自有 PC、寄存器与栈。
- 线程通常共享进程的地址空间、代码和打开的文件等资源。
- 在多核上，同一进程的不同线程可以并行运行。

进程侧重资源与隔离；线程侧重执行流。

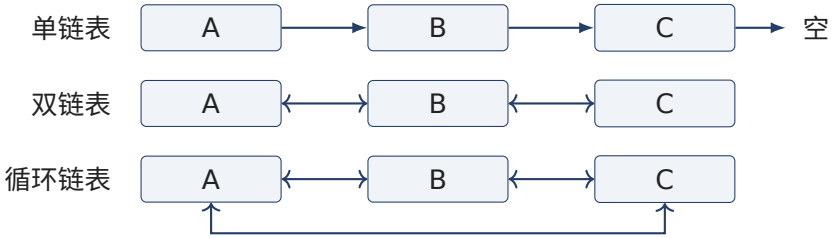
资料：[1] Operating System Concepts，第 3 章

谁获得下一个 CPU 时间片？

- **调度器（Scheduler）**从可运行任务中选择下一个执行者。
- **就绪队列**：组织具备运行条件、等待 CPU 的任务。
- **等待队列**：按 I/O、消息等事件组织等待者。
- 时间片用完可能回到就绪；等待事件时离开 CPU，事件完成后重新就绪。
- 调度同时考虑响应、公平与资源利用，具体算法在后续章节展开。

资料：[1] Operating System Concepts，第 3 章

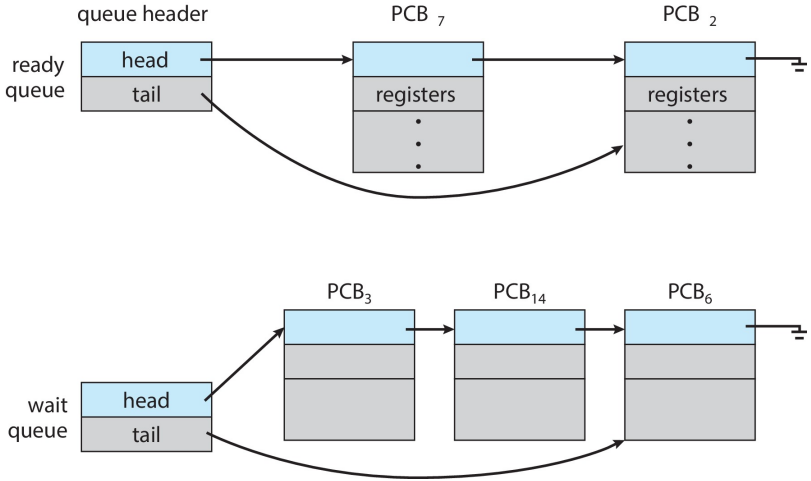
队列怎样连接 PCB?



- 队列保存任务之间的连接关系，便于入队、出队和遍历。
- Linux 常用嵌入对象中的链表节点，把对象组织起来。
- **用什么数据结构组织任务，与先调度谁，是两个问题。**

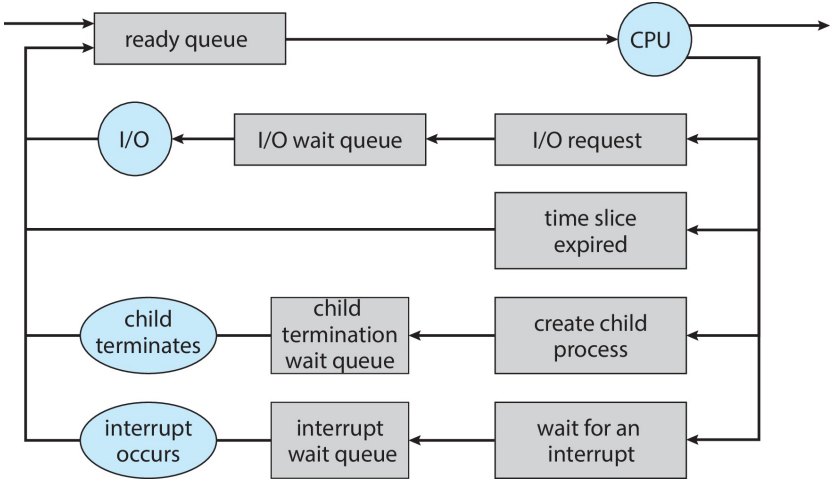
资料：[6] Linux：Linked Lists [1] Operating System Concepts，第 3 章

就绪队列与等待队列



资料：[1] Operating System Concepts，第 3 章

进程在队列之间移动



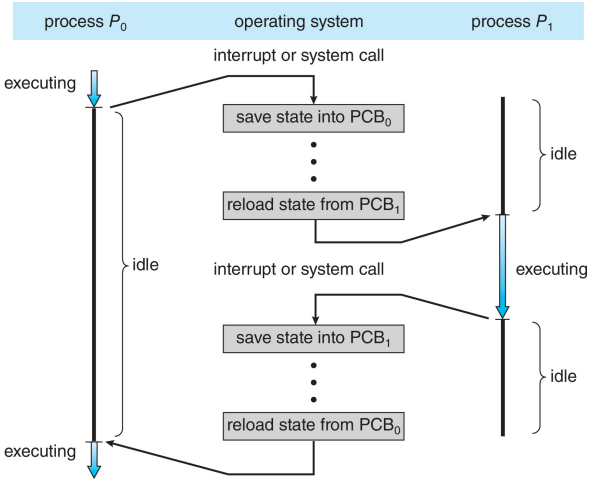
资料：[1] Operating System Concepts，第 3 章

切换进程时要保存什么?

- 暂停当前任务，保存恢复执行所需的 **上下文（Context）**。
- 选出下一任务，切换相关内核状态和必要的地址空间信息。
- 恢复新任务的寄存器等现场，从上次停下的位置继续。
- **进入内核不等于切换进程**：系统调用也可以处理后返回原任务。

资料：[1] Operating System Concepts，第 3 章

CPU 怎样从一个进程切到另一个?



资料：[1] Operating System Concepts，第 3 章

是否应该避免所有上下文切换？

- 切换会花时间；缓存和地址转换状态变化也可能带来后续开销。
- 等待 I/O 时切换，可以让其他任务利用 CPU。
- 时间片切换让交互任务及时响应，也防止一个任务长期独占 CPU。
- 应减少**没有必要的频繁切换**，同时保留并发、公平与响应能力。

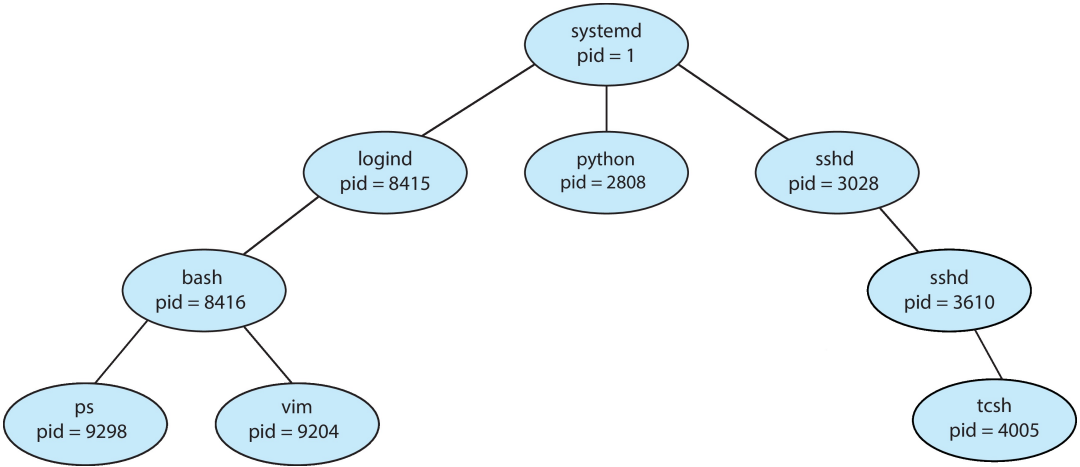
资料：[1] Operating System Concepts，第 3 章

进程从哪里来?

- 系统初始化会建立早期进程，已有进程还可以创建新进程。
- 创建者是**父进程**，新进程是**子进程**；关系可形成进程树。
- 系统通过 **PID（Process Identifier）**等信息识别和管理进程。
- 父子进程可并发推进，也可以通过等待建立先后关系。
- 地址空间与资源如何复制、共享或替换，取决于创建接口。

资料：[1] Operating System Concepts，第 3 章

进程树



资料：[1] Operating System Concepts，第 3 章

fork 为什么会返回两次?

```
pid_t pid = fork();
if (pid < 0) { /* creation failed */ }
else if (pid == 0) { /* child */ }
else { /* parent: pid is the child's PID */ }
```

- 一次成功调用创建一个子进程，父子各自从调用后继续执行。
- 子进程得到 `0`，父进程得到子进程 PID；失败时只有父进程得到 `-1`。
- 父子谁先继续执行，**没有默认顺序保证**。

资料：[2] fork(2)

fork 后，哪些东西相同?

内容	父子进程之间的关系
普通变量	初始值相同；此后通常各自修改
虚拟地址空间	各自独立；Linux 常用写时复制实现
继承的文件描述符	指向同一打开文件描述，可共享文件偏移
PID 与执行进度	PID 不同，之后可独立推进

复制变量的值，不等于共享同一个可写变量。

资料：[2] fork(2)　[17] stdio(3)

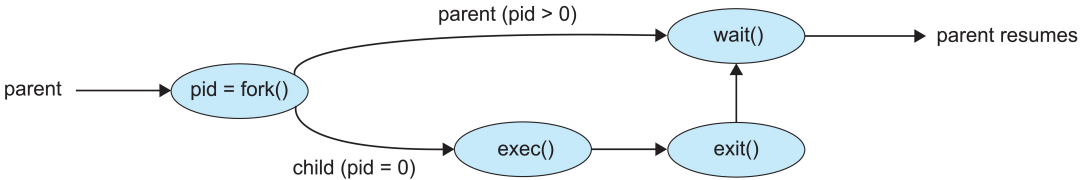
exec 和 wait 各做什么?

- **fork**：创建新进程。
- **exec 系列**：在当前进程中装入新程序，成功后不返回原程序。
- **wait / waitpid**：等待子进程状态变化，并取得、回收退出信息。
- `exec` 通常保留进程 PID，改变的是正在执行的程序。

Shell 常见流程：创建子进程 → 子进程执行命令 → 父进程等待或继续。

资料：[2] fork(2)　[3] execve(2)　[4] wait(2)

fork、exec 与 wait



资料：[1] Operating System Concepts，第 3 章

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    printf("[0] Hello from %d\n", pid); // 0

    if (pid == 0) {
        printf("[1] [Child] before exec\n"); // 1
        execlp("echo", "echo", "\033[31m[2] Hello from the echo command!\033[0m", NULL); // 2
        printf("[3] [Child] after exec\n"); // 3
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("[4] [Parent] child exited with code %d\n", WEXITSTATUS(status)); // 4
    }
}
```

运行这个程序时，打印语句的执行顺序可能是怎么样的？

进程怎样结束？

- 正常完成：返回 `main` 或调用 `exit`，把退出状态交给系统。
- 异常终止：未处理的致命信号等使进程结束。
- 其他进程可在权限允许时发送信号，例如请求终止。
- 内核释放大部分资源，保留父进程取得退出结果所需的信息。
- 父进程应检查等待结果，再判断是正常退出还是被信号终止。

资料：[4] wait(2)　[19] kill(2)　[17] stdio(3)

怎样取得子进程的退出结果?

```
pid_t r;
do { r = waitpid(pid, &status, 0); }
while (r < 0 && errno == EINTR);
if (r < 0) { /* handle error */ }
else if (WIFEXITED(status))
    printf("exit=%d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("signal=%d\n", WTERMSIG(status));
```

- `status` 是编码后的状态，不能直接当作退出码。
- 先判断 **WIFEXITED**，再使用 **WEXITSTATUS**。
- `waitpid` 可以指定要等待的子进程，完整示例见 `examples/`。

资料：[4] wait(2)

僵尸进程与孤儿进程

情况	含义	后续处理
僵尸（Zombie）	子进程已结束，退出信息尚未回收	父进程等待并回收等
孤儿（Orphan）	原父进程先结束，子进程仍存在	Linux 中被重新托管

- **父进程没有调用 wait，不代表仍在运行的子进程就是僵尸。**
- 父进程退出后，子进程并不一定跟着退出；行为取决于系统和管理策略。

资料：[4] wait(2)

手机为什么限制后台进程?

- 屏幕上看不见的应用，仍可能消耗 CPU、内存和电量。
- 系统根据任务用途、用户可见性和资源压力，限制后台执行或回收进程。
- Android 区分前台、可见、服务、缓存等重要性情形，低重要性进程更容易被回收。
- iOS 提供受系统调度和条件限制的后台任务机制。
- 应用需要保存可恢复状态；**在后台，不等于一直运行**。

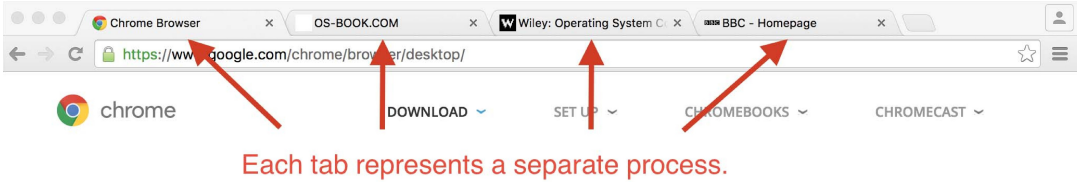
资料：[14] Android：Process Lifecycle　[15] Apple：Background Tasks

浏览器为什么采用多进程?

- **浏览器进程**：组织窗口、用户交互与跨进程协调。
- **渲染进程**：执行页面相关工作，在沙箱约束下运行。
- **GPU、网络等服务进程**：承担专门任务，具体划分随实现变化。
- 收益：故障隔离、安全边界与并行处理；代价：更多资源和 IPC。
- **每个标签页一个进程**：用这一简化模型理解不同页面之间的隔离。

资料：[16] Chromium：Multi-process Architecture

Chrome 标签页与进程



资料：[1] Operating System Concepts，第 3 章

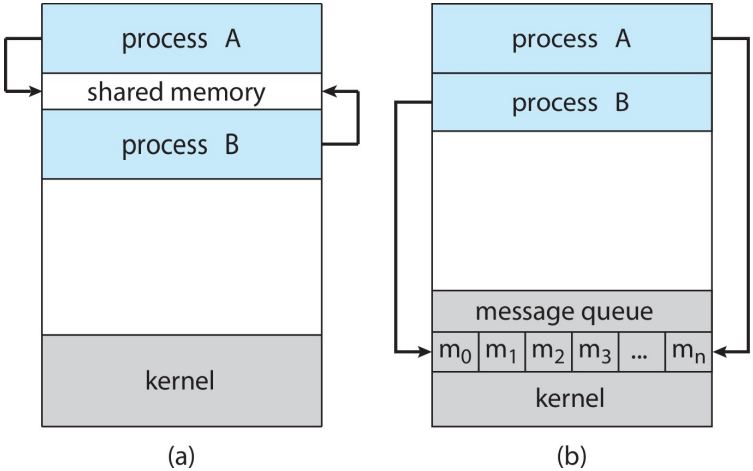
进程为什么需要通信?

- **信息共享**：一个进程生成数据，另一个进程使用。
- **计算协作**：把工作分给多个执行者，再汇总结果。
- **模块化与隔离**：界面、计算和服务分别运行。
- **IPC（Interprocess Communication）**：让不同进程交换数据并协调行动。

进程彼此隔离；要合作，就需要约定怎样共享或传递信息。

资料：[1] Operating System Concepts，第 3 章

两种进程通信模型



资料：[1] Operating System Concepts，第 3 章

共享内存与消息传递

比较	共享内存	消息传递
数据在哪里	多个进程共同映射的区域	发送缓冲、通道与接收端
程序如何使用	读写共享数据结构	按接口发送、接收
需要约定什么	布局、访问顺序与同步	格式、寻址、等待与容量
适用直觉	本机大量数据的协作	明确边界的本机或跨机通信

两种方式都需要处理生命周期、权限与并发。

资料：[1] Operating System Concepts，第 3 章

生产者与消费者

- **生产者（Producer）**生成数据，**消费者（Consumer）**处理数据。
- 中间缓冲区暂存已产生、尚未处理的数据。
- 无数据时消费者需要等待；缓冲区满时生产者也可能等待。
- 两者速度可以不同，缓冲区吸收一段时间内的差异。

例如：摄像头产生视频帧，编码进程取出并处理。

资料：[1] Operating System Concepts，第 3 章

生产者与消费者的协作



有界缓冲区与无界缓冲区

前面的生产者、消费者共用一个缓冲区；有无容量限制决定了双方什么时候需要等待。



- **有界缓冲区**容量固定；例如容量 4，当前放了 2 项。
- 空时不能取，满时不能放；条件改变后唤醒等待的一方。
- **无界缓冲区**模型忽略容量限制，消费者仍可能等数据。
- 读写位置、数据和数量必须保持一致，需要同步机制保证。

资料：1. Operating System Concepts，第 3 章。

共享内存：从协作需求到实现

缓冲区说明了生产者和消费者怎样协作。接下来用共享内存实现一次数据传递。

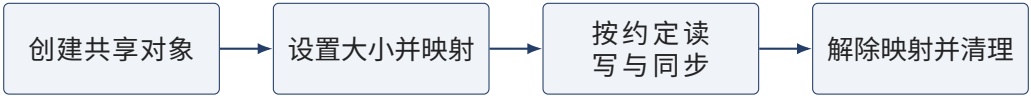
- **建立共享区域**：让两个进程访问同一份数据。
- **规定访问顺序**：先写完，再通知消费者读取。
- **完成一次通信**：父进程发送字符串，子进程等待并读取。

共享内存提供数据存放的位置；同步机制保证双方按正确顺序访问。

资料：1. Operating System Concepts，第 3 章。

建立与释放共享内存

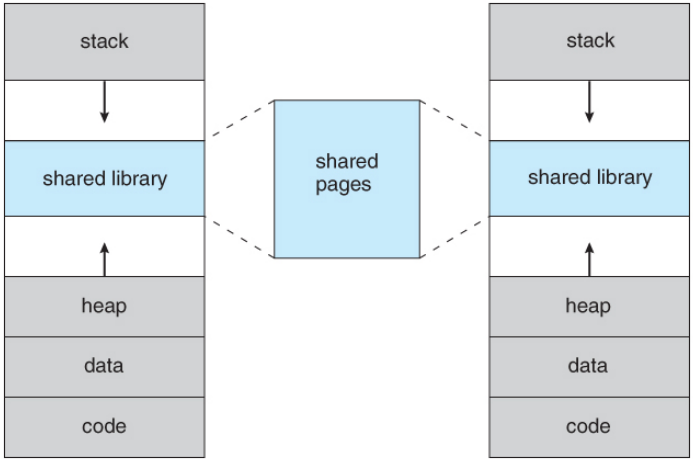
先建立双方都能访问的区域，再安排数据读写，最后释放通信资源。



- POSIX 常见接口：`shm_open`、`ftruncate`、`mmap`、`shm_unlink`。
- 用 **MAP_SHARED** 建立共享映射；各进程映射地址可以不同。
- 共享的是映射区域，不是进程的全部堆、栈或全局变量。
- 共享数据中宜保存索引或偏移，不直接假设另一进程能使用本地指针。

资料：9. shm_open(3)；20. mmap(2)。

两个地址空间共享同一组页



资料：1. Operating System Concepts，第 3 章。

共享内存中的同步

映射同一份数据只解决了访问位置，还需要保证写入、通知和读取的顺序。

- 生产者不能先宣布“有数据”，再把数据写完整。
- 消费者不能在生产者写到一半时读取。
- 需要**互斥与顺序保证**，例如互斥锁、信号量等。
- 一个“数据就绪”标志，也必须满足跨进程同步要求。

共享内存让双方访问同一份数据；同步保证访问时机和顺序正确。

资料：10. sem_overview(7)；20. mmap(2)。

共享内存示例：准备数据与信号量

父进程先准备共享区域，再调用 fork；子进程继承映射，双方按下一页的代码收发数据。

```
struct message {
    sem_t ready;
    char text[128];
};
int fd = shm_open("/ipc_example", O_CREAT | O_EXCL | O_RDWR, 0600);
ftruncate(fd, sizeof(struct message));
struct message *shared = mmap(NULL, sizeof *shared,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
close(fd);
sem_init(&shared->ready, 1, 0);
```

- `text` 保存字符串，`ready` 用于通知数据已写完。
- `sem_init` 的 `1` 表示进程间共享，`0` 是初值：消费者起初需要等待。

资料：9. shm_open(3)；10. sem_overview(7)。

共享内存示例：写入、通知与读取

共享区域已经准备好，父进程写入字符串，子进程收到通知后读取。

```
/* parent: producer */
snprintf(shared->text, sizeof shared->text, "hello");
sem_post(&shared->ready);

/* child: consumer */
sem_wait(&shared->ready);
printf("%s\n", shared->text);
```

- 父进程**先写完整，再通知**；sem_post 表示数据已就绪。
- 子进程调用 sem_wait；成功返回后才读取，因此不会读到写了一半的数据。
- 这里只传递一次字符串；重复发送还需要防止上一条数据被过早覆盖。

资料：9. shm_open(3)；10. sem_overview(7)。

消息传递：寻址、等待与缓冲

共享内存由双方直接读写同一块区域。消息传递则通过发送、接收接口交换数据。

- **寻址**：指定接收进程，或指定共同使用的邮箱。
- **等待**：消息尚未到达，或暂时无法发送时，调用怎样返回？
- **缓冲**：消息能否暂存，容量不足时怎样处理？

接下来分别讨论这些约定，再看管道和 Socket 等具体机制。

资料：1. Operating System Concepts，第 3 章。

消息传递的基本约定

通信双方首先要约定接收对象、消息格式，以及发送和接收的行为。

- **发给谁**：指定进程、邮箱或其他端点。
- **传什么**：固定或可变大小的消息，约定格式和长度。
- **怎样连接**：点对点或多参与者，单向或双向。
- **是否等待**：发送、接收何时返回。
- **存在哪里**：消息是否缓冲，容量满时怎样处理。

资料：1. Operating System Concepts，第 3 章。

直接通信与间接通信

先确定接收对象：是直接指定进程，还是把消息发往一个邮箱？

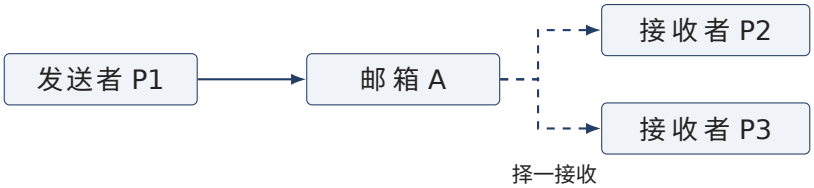
方式	指定什么	示例
直接通信	通信对方	send(P, message)
间接通信	共同使用的邮箱或端口	send(A, message)

- 直接寻址把通信与对方身份绑定；教材的对称模型为每对通信进程建立链路。
- 邮箱把接收位置与具体接收者分开，可有多个参与者。
- 邮箱通常需要创建、收发、删除；两个进程之间可以使用多个邮箱。

资料：1. Operating System Concepts，第 3 章。

共享邮箱中的消息分配

一个邮箱可以供多个进程接收，但每条消息最终由谁取得，需要明确规则。



- 共享一个邮箱，不意味着每个接收者都会得到一份消息。
- 接口可以限制为一对一，或限制同一时刻只有一个接收者。
- 也可以按规则选择一个接收者；是否通知发送者由协议约定。
- 若要广播给所有参与者，需要明确约定广播或发布订阅规则。

资料：1. Operating System Concepts，第 3 章。

阻塞与非阻塞通信

确定通信对象后，还要规定没有数据或无法发送时，调用是否等待。

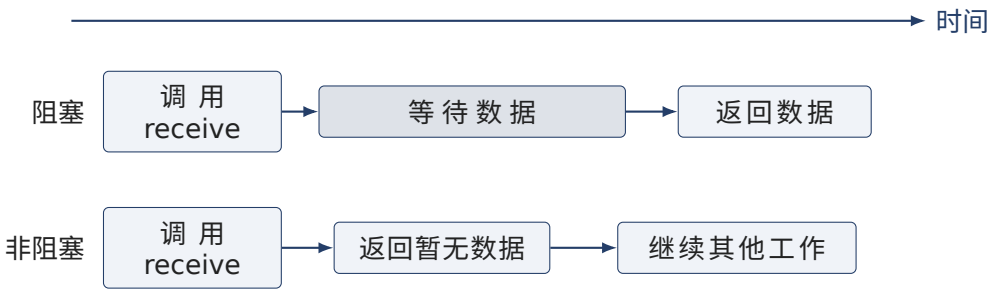
操作	阻塞方式	非阻塞方式
接收	无可用消息时等待	无消息时立即返回，报告暂无数据
发送	未达到接口完成条件时等待	不能立即发送时立即返回，报告当前状态

- 零容量的同步交接，可要求双方都准备好后再传递。
- 有缓冲的发送可能在入队后就返回，**不代表对方已处理消息**。
- 具体返回值与完成条件，要以接口约定为准。

资料：1. Operating System Concepts，第 3 章。

阻塞接收与非阻塞接收

以暂时没有数据为例，比较一次阻塞接收和非阻塞接收的返回时机。



资料：1. Operating System Concepts，第 3 章。

通信通道的缓冲容量

消息可以先进入通道等待接收，但能否继续发送取决于缓冲容量。

- **零容量**：不暂存消息，发送与接收需要配合交接。
- **有界容量**：可暂存有限消息；满时阻塞、报错或丢弃由接口决定。
- **无界容量模型**：忽略满的情况，真实系统仍受资源限制。
- 容量影响等待、内存占用与排队延迟。

发送速度持续超过接收速度时，缓冲区仍然可能被填满。

资料：1. Operating System Concepts，第 3 章。

具体通信机制：管道与 Socket

前面讨论的是消息传递的一般约定。接下来比较程序中常用的通信接口。

机制	如何找到通信对象	常见用途
普通管道	继承或传递文件描述符	本机相关进程之间传数据
命名管道（FIFO）	文件系统路径	本机进程按名称通信
Socket	本机路径，或 IP 地址与端口等	本机或跨机器通信

管道和 TCP 都传输字节流，程序还要约定怎样区分一条条消息。

资料：7. pipe(7)；11. socket(7)。

普通管道与文件描述符

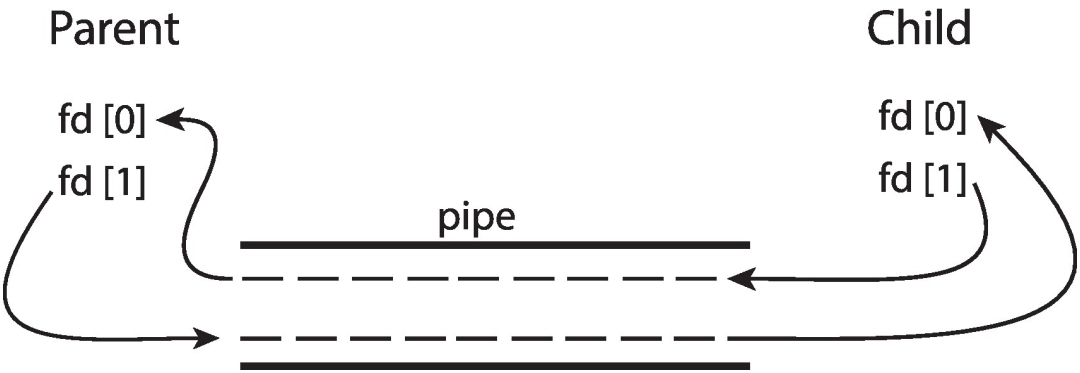
先看本机通信：pipe 创建一个字节流通道，并返回读端、写端两个文件描述符。

```
int fd[2];
pipe(fd);   /* fd[0]: read end; fd[1]: write end */
```

- Linux 普通管道是**单向字节流**，没有应用消息边界。
- 先创建管道再 `fork`，父子便可继承读写端。
- 常用于相关进程；关键是能访问同一管道端点。
- 双向通信通常需要两条管道，或使用支持双向通信的其他机制。

资料：7. pipe(7)。

父子进程共享管道端点



资料：1. Operating System Concepts，第 3 章。

管道示例：父进程写，子进程读

父、子进程各自只保留需要的端点：父进程发送，子进程读取到 EOF。

父进程

```
close(fd[0]);
write(fd[1], "hello", 5);
close(fd[1]);
waitpid(pid, NULL, 0);
```

子进程

```
char buf[64]; ssize_t n;
close(fd[1]);
while ((n = read(fd[0], buf,
                 sizeof buf)) > 0) {
    fwrite(buf, 1, (size_t)n, stdout);
}
close(fd[0]);
```

- `fd` 是前面 `pipe` 返回的描述符，`pid` 是父进程得到的子进程编号。
- 子进程用 `fwrite` 输出实际读到的 `n` 个字节；一次写入不一定对应一次读取。
- 实际代码需检查返回值：短读写要继续处理，`EINTR` 表示调用被信号中断，必要时重试。

资料：7. pipe(7)。

管道何时返回 EOF?

上一页关闭写端不仅是释放资源，也是在告诉读端：后面不会再有数据。

- 管道中仍有数据：`read` 可以取走已有字节。
- 管道已空但**仍有写端打开**：阻塞读取通常继续等待。
- 管道已空且**所有写端都关闭**：`read` 返回 `0`，表示 EOF。
- 子进程若忘关自己继承的写端，也可能等不到 EOF。

写端存在的判断，涉及所有持有该端点的进程。

资料：7. pipe(7)。

命名管道（FIFO）

没有继承同一组管道描述符的进程，可以通过 FIFO 路径找到同一通信对象。

- **FIFO（Named pipe）**有文件系统路径，不相关进程也能按名字打开。
- 路径用于找到通信对象；传输的数据不作为普通文件内容存盘。
- Linux FIFO 的 I/O 语义类似普通管道，不能默认当成全双工通道。
- Windows 命名管道可以配置不同方向与模式，应区分平台。
- 多进程使用时，还需约定谁读、谁写以及怎样划分消息。

资料：8. fifo(7)；7. pipe(7)；21. Microsoft：Named Pipes。

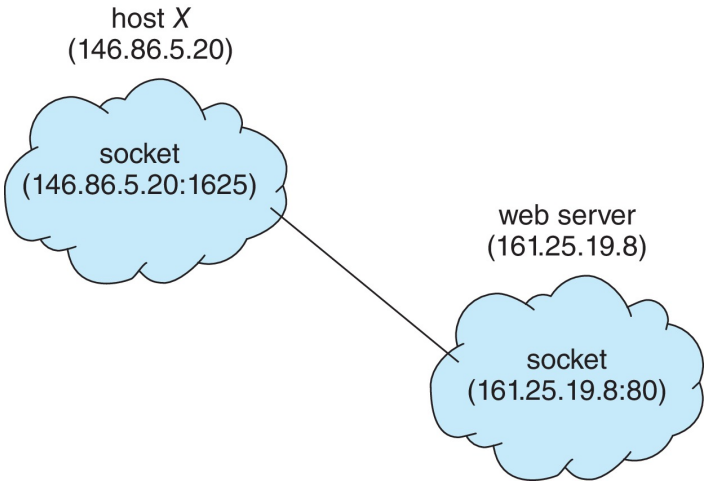
Socket 与通信端点

需要跨机器通信时，可以使用 Socket；它也支持本机进程间通信。

- **Socket** 是通信端点；可用于本机进程或跨机器通信。
- 对 TCP/IP，可用 IP 地址与端口定位端点，还需区分 TCP、UDP 等协议。
- 服务端在约定地址和端口等待，客户端发起连接或发送数据。
- `127.0.0.1` 是 IPv4 回环地址，用来访问本机。
- 一个 TCP 连接由两端的地址、端口等共同区分。

资料：11. socket(7)。

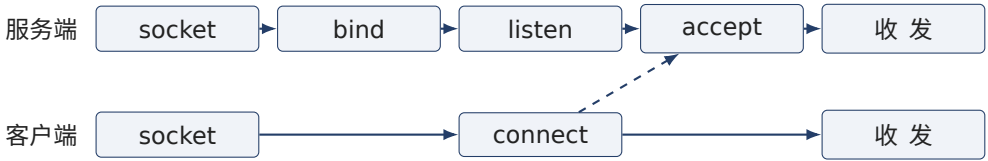
客户端与服务端的端点



资料：1. Operating System Concepts，第 3 章。

TCP 的连接与数据传输

下面用 TCP 展示双方如何建立连接，以及数据传输时还需要哪些约定。



- `accept` 返回与某客户端通信的已连接 socket，监听 socket 继续存在。
- TCP 提供有序字节流；应用仍需规定消息长度、分隔符或其他边界。
- `send` 成功不代表对方应用已完成业务处理。
- 例如双方约定每条文本消息以换行符结束，接收端累积读取到换行符再处理。

资料：11. socket(7)。

从 Socket 通信到远程过程调用

Socket 提供收发数据的接口，但程序还要解释数据，并决定执行什么操作。

以客户端请求服务端执行 add(2, 3) 为例：

- 把操作名称和参数编码成请求，发送给服务端。
- 服务端解码并执行加法，把结果 5 编码后返回。
- 客户端接收结果，还要处理连接失败或超时。

RPC 把这些步骤封装为类似函数调用的接口，后面讨论调用流程、数据格式和重试。

资料：12. RFC 5531：RPC。

远程过程调用（RPC）

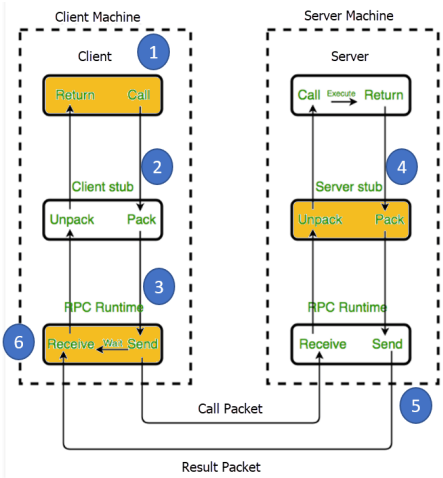
RPC 将请求编码、数据传输和结果解码等步骤封装起来，供程序发起远程调用。

- **RPC（Remote Procedure Call）**把对远程服务的请求封装为类似本地函数调用的接口。
- 客户端 **stub（存根）**把参数编码，发送请求。
- 服务端解码参数，调用相应操作，再编码并返回结果。
- 服务地址可以由配置或发现机制确定；连接和请求仍可能失败。

调用形式像本地函数，执行过程还包括数据传输、编码和解码。

资料：12. RFC 5531：RPC。

一次 RPC 的请求与返回



资料：1. Operating System Concepts，第 3 章；12. RFC 5531：RPC。

RPC 的参数编码与数据格式

请求和返回值需要共同的数据格式，才能在不同机器上被正确解释。

- 不同机器可能使用不同的字节序、类型宽度和数据布局。
- **序列化 / 编组（Serialization / Marshalling）**把数据编码为约定格式。
- 接收者按同一格式解码；不能直接把本机指针发过去当作远端地址。
- **XDR（External Data Representation）**是一种外部数据表示标准。
- RPC 还需约定参数类型、接口版本和错误表示。

资料：13. RFC 4506：XDR；12. RFC 5531：RPC。

RPC 超时与重试

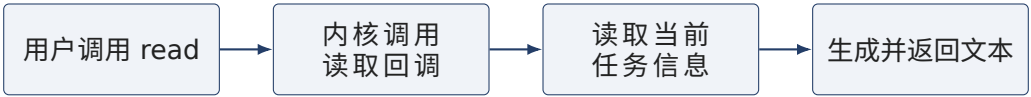
数据格式一致并不保证调用成功；超时后，客户端可能不知道服务端是否已经执行。

- 超时可能是请求没到，也可能是**已执行但回复丢失**。
- 直接重试，可能让扣款、下单等操作执行两次。
- **至多一次（At most once）**通过请求标识等避免重复执行，但调用可能没有完成。
- **至少一次（At least once）**在相应故障条件下通过重试争取成功，同一操作可能重复执行。
- 幂等操作重复执行效果相同；非幂等操作需要请求标识、去重等机制，RPC 本身不保证恰好一次。

资料：12. RFC 5531：RPC。

选讲：通过 /proc 获取内核信息

最后回到进程的内核表示：用户程序通过 /proc 文件接口读取内核提供的信息。

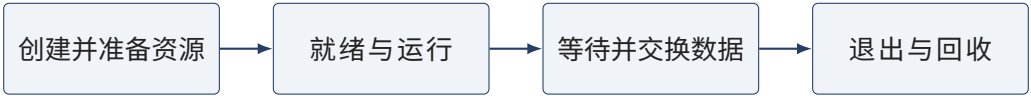


- 例如用户执行 `cat /proc/self/status`，内核返回这个读取进程的状态信息。
- 自定义 `/proc` 条目也可用读取回调取得 `current` 的进程名和 PID。
- `seq_file` 帮助生成文本，`proc_ops` 指定文件操作回调。
- 内核模块需要按目标内核版本构建；此处只说明接口工作过程。

资料：18. Linux：seq_file；5. proc_pid_status(5)；22. Linux v6.12：proc_fs.h。

本章小结

进程的资源、调度、创建退出和通信，共同构成了本章的内容。



- **PCB 与地址空间**让内核能管理和恢复一个进程。
- **调度与上下文切换**让多个进程轮流使用 CPU。
- **fork / exec / wait** 控制创建、程序替换和退出回收。
- **IPC** 让隔离的进程能够协作；通信还需要格式和同步约定。

资料：1. Operating System Concepts，第 3 章。

课堂练习

- I/O 完成后，进程为什么通常先回到就绪状态?
- fork 后子进程修改普通变量，父进程是否一定能看到?
- exec 成功后，为什么原程序后面的打印没有执行?
- 管道中已经没有数据，read 为什么仍可能等待?
- RPC 调用超时，是否能认定服务端没有执行?

参考资料

[1] Silberschatz, Galvin, Gagne (2018) —Operating System Concepts, 10th ed., Chapter 3: Processes

[2] Linux man-pages —fork(2): create a child process

[3] Linux man-pages —execve(2): execute program

[4] Linux man-pages —wait(2): wait for process to change state

[5] Linux man-pages —proc_pid_status(5): memory usage and status information

[6] The Linux Kernel documentation —Linked Lists in Linux

[7] Linux man-pages —pipe(7): overview of pipes and FIFOs

[8] Linux man-pages —fifo(7): first-in first-out special file, named pipe

[9] Linux man-pages —shm_open(3): create/open or unlink POSIX shared memory objects

[10] Linux man-pages —sem_overview(7): overview of POSIX semaphores

[11] Linux man-pages —socket(7): Linux socket interface

[12] R. Thurlow (2009) —RFC 5531: RPC, Remote Procedure Call Protocol Specification Version 2

[13] M. Eisler (2006) —RFC 4506: XDR, External Data Representation Standard

[14] Android Developers —Processes and app lifecycle

[15] Apple Developer Documentation —Background Tasks

[16] The Chromium Projects —Multi-process Architecture

[17] Linux man-pages —stdio(3): standard input/output library functions

[18] The Linux Kernel documentation —The seq_file Interface

[19] Linux man-pages —kill(2): send signal to a process

[20] Linux man-pages —mmap(2): map or unmap files or devices into memory

[21] Microsoft Learn (2021) —Named Pipes

[22] Linux v6.12 —include/linux/proc_fs.h: proc_ops and proc_create